

practical1

AnagramCheck.java

```
// Import Scanner: used to read text input typed on the keyboard.
import java.util.Scanner;

/**
 * Checks whether two user-provided strings are anagrams of each other
 * (contain the same characters with the same frequencies, case-insensitive).
 */
public class AnagramCheck {
    public static void main(String[] args) {
        // Create a Scanner object that reads from System.in (the keyboard).
        Scanner sc = new Scanner(System.in);

        // Ask the user to type the first text.
        System.out.println("Please enter text 1 : ");
        // Read the first word and convert it to lowercase so the check ignores letter case.
        String text1 = sc.next().toLowerCase();
        // Ask the user to type the second text.
        System.out.println("Please enter text 2 : ");
        // Read the second word and convert it to lowercase.
        String text2 = sc.next().toLowerCase();

        // flag records whether the two texts are anagrams, assumed true until a mismatch is found.
        boolean flag = true;
        // count1 will count occurrences of a character in text1, reset for each character.
        int count1 = 0;
        // count2 will count occurrences of the same character in text2, reset for each character.
        int count2 = 0;
        // Convert text1 into a char array so we can visit each character.
        char[] arr1 = text1.toCharArray();
        // Convert text2 into a char array so we can visit each character.
        char[] arr2 = text2.toCharArray();

        // Quick length check – anagrams must have the same number of characters
        if (arr1.length != arr2.length) {
            // Different lengths mean they cannot possibly be anagrams.
            System.out.println("Not an anagram!!");
        } else {
            // For each character in text1, count its occurrences in both strings
            // Outer loop: pick each character of text1 one at a time.
            for (int a = 0; a < arr1.length; a++) {
                // Reset text1's count for this new character.
                count1 = 0;
                // Reset text2's count for this new character.
                count2 = 0;
                // Count how many times arr1[a] appears in arr1
                // Inner loop: scan all of arr1 to count matches with the chosen character.
                for (int b = 0; b < arr1.length; b++) {
                    if (arr1[a] == arr1[b]) {
                        count1++;
                    }
                }
                // Count how many times arr1[a] appears in arr2
                // Inner loop: scan all of arr2 to count matches with the same character.
                for (int b = 0; b < arr2.length; b++) {
                    if (arr1[a] == arr2[b]) {
                        count2++;
                    }
                }
            }
        }
    }
}
```

```

    }
}
// If frequencies don't match, not an anagram
// The character appears a different number of times in each text.
if (count1 != count2) {
    // Record that the texts are not anagrams.
    flag = false;
    // Stop checking the remaining characters since the answer is already known.
    break;
}
}

// If every character had matching frequencies, the texts are anagrams.
if (flag) {
    System.out.println("Is an anagram!");
    // Otherwise at least one character frequency differed.
} else {
    System.out.println("Not an anagram!!");
}
}
// Close the Scanner to release the keyboard resource.
sc.close();
}
}

```

ArmstrongCheck.java

```
// Import Scanner: used to read text input typed on the keyboard.
import java.util.Scanner;
// Import Math: provides the pow() method used to raise a digit to the power of the number of digits.
import java.lang.Math;

/**
 * Checks whether a given number is an Armstrong number.
 * An Armstrong number is one where the sum of its digits each raised to the
 * power of the total number of digits equals the original number.
 */
public class ArmstrongCheck {
    public static void main(String[] args) {
        // Create a Scanner object that reads from System.in (the keyboard).
        Scanner sc = new Scanner(System.in);

        // Read the whole number typed by the user and store it in num.
        int num = sc.nextInt();
        // tempc keeps a copy of num so we can count digits without losing the original; tempa is another copy for
        // extracting digits later.
        int tempc = num, tempa = num;
        // sum will hold the total of every digit raised to the power of the digit count, started at 0.
        int sum = 0;
        // count will track how many digits the number has, started at 0.
        int count = 0;

        // Count the number of digits
        while (tempc > 0) {
            // Each digit removed from the number means one more digit, so increment the counter.
            count++;
            // Remove the last digit by integer-dividing by 10.
            tempc /= 10;
        }

        // Extract each digit and add digit^count to the sum
        while (tempa > 0) {
            // Get the last digit using the modulo operator (% 10 gives the units place).
            int digit = tempa % 10;
            // Raise this digit to the power of the digit count and add it to the running sum.
            sum += Math.pow(digit, count);
            // Remove the last digit by integer-dividing by 10.
            tempa /= 10;
        }

        // If the sum of the powered digits equals the original number, it is an Armstrong number.
        if (sum == num)
            System.out.println(num + " is an armstrong number!");
        // Otherwise the number is not an Armstrong number.
        else
            System.out.println(num + " is not an armstrong number!");
        // Close the Scanner to release the keyboard resource.
        sc.close();
    }
}
```

FibonacciSeries.java

```
// Import Scanner: used to read text input typed on the keyboard.
import java.util.Scanner;

/**
 * Generates and prints the Fibonacci series up to a user-specified number of
 * terms. Each term is the sum of the two preceding terms (starting with 0, 1).
 */
public class FibonacciSeries {
    public static void main(String[] args) {
        // Create a Scanner object that reads from System.in (the keyboard).
        Scanner sc = new Scanner(System.in);

        // Ask the user how many terms of the series they want.
        System.out.println("Please enter the range of the series : ");
        // Read the range as an integer.
        int range = sc.nextInt();

        // a and b hold the previous two terms; the series starts with 0 and 1.
        int a = 0, b = 1;
        // Print the first term (0) and start the series output.
        System.out.println("Series : \n" + a);

        // Generate subsequent terms by adding the previous two
        // Loop once for every remaining term in the requested range.
        for (int i = 1; i <= range; i++) {
            // Print the current second term (b) before shifting.
            System.out.println(b);
            // c is the sum of the previous two terms.
            int c = a + b;
            // Shift a forward to hold the old b.
            a = b;
            // Shift b forward to hold the newly computed term.
            b = c;
        }
    }
}
```

LeapYearCheck.java

```
// Import Scanner: used to read text input typed on the keyboard.
import java.util.Scanner;

/**
 * Checks whether a given year is a leap year based on the standard rule:
 * divisible by 400, or divisible by 4 but not by 100.
 */
public class LeapYearCheck {
    public static void main(String[] args) {
        // Create a Scanner object that reads from System.in (the keyboard).
        Scanner sc = new Scanner(System.in);
        // Ask the user to type the year they want to check.
        System.out.println("Please enter the year to be checked : ");
        // Read the year typed by the user as an integer.
        int year = sc.nextInt();

        // Leap year if divisible by 400, or divisible by 4 but not 100
        if (year % 400 == 0 || (year % 4 == 0 && year % 100 != 0))
            System.out.println(year + " is a leap year!");
        // Otherwise (does not satisfy either rule) it is not a leap year.
        else
            System.out.println(year + " is not a leap year!");
    }
}
```



```
        break;
// Digit '9' is converted to the word "Nine".
case '9':
    System.out.print("Nine ");
    break;
// If the character is not a digit, report it as invalid.
default:
    System.out.print("Invalid ");
    }
    }
}
```

OddCount.java

```
/**
 * Counts and prints the number of odd integers between 1 and 500 (inclusive).
 */
public class OddCount {
    public static void main(String[] args) {
        // Start the counter at 0 because no odd numbers have been counted yet.
        int count = 0;

        // Check each number from 1 to 500; increment count for odd numbers:
        // the loop below visits every whole number between 1 and 500 (inclusive).
        for (int i = 1; i <= 500; i++) {
            // A number is odd when dividing it by 2 leaves a remainder (i % 2 != 0).
            if (i % 2 != 0)
                // The current number is odd, so increase the counter by 1.
                count++;
        }

        // Print the final total number of odd numbers found in the range.
        System.out.println("The number of odd numbers between 1 and 500 is : " + count);
    }
}
```

Palindrome.java

```
// Import Scanner: used to read text input typed on the keyboard.
import java.util.Scanner;

/**
 * Reads a string from the user and checks whether it is a palindrome
 * (reads the same forwards and backwards, case-insensitive).
 */
public class Palindrome {
    public static void main(String[] args) {
        // Create a Scanner object that reads from System.in (the keyboard).
        Scanner sc = new Scanner(System.in);
        // Ask the user to type in a word.
        System.out.println("Please enter your string : ");
        // Read the word typed by the user.
        String text = sc.next();
        // Convert it to lowercase so the palindrome check ignores letter case.
        text = text.toLowerCase();

        // Assume the word is a palindrome; we will set this to false if a mismatch is found.
        boolean isPalindrome = true;

        // Compare characters from both ends moving inward
        // Only need to check up to the middle of the word; each pair is compared once.
        for (int i = 0; i <= text.length() / 2; i++) {
            // Compare the i-th character from the left with the matching character from the right.
            if (text.charAt(i) != text.charAt(text.length() - i - 1)) {
                // The characters differ, so the word is not a palindrome.
                isPalindrome = false;
                // Stop checking further characters since we already know the answer.
                break;
            }
        }

        // If no mismatches were found, the word reads the same forwards and backwards.
        if (isPalindrome)
            System.out.println(text + " is a palindrome!!");
        // Otherwise it is not a palindrome.
        else
            System.out.println(text + " is not a palindrome!!");
    }
}
```

PrimeNumbersGeneration.java

```
/**
 * Prints all prime numbers between 1 and 500 using a brute-force
 * divisibility check for each number.
 */
public class PrimeNumbersGeneration {
    public static void main(String[] args) {
        // n is the upper limit of the range we will search for primes.
        int n = 500;
        // Print a heading stating which range is being searched.
        System.out.println("Prime numbers between 1 and " + n + " are: ");

        // Check each number from 1 to n for primality
        // Outer loop: test every number in the range one by one.
        for (int i = 1; i <= n; i++) {
            // Assume the current number is prime until a divisor is found.
            boolean isPrime = true;

            // A prime has no divisors other than 1 and itself
            // Try every possible divisor from 2 up to half the number (i/2 is enough).
            for (int j = 2; j <= i / 2; j++) {
                // If j divides i exactly, then i is not prime.
                if (i % j == 0) {
                    // Mark the number as not prime.
                    isPrime = false;
                    // No need to keep checking; stop the divisor loop.
                    break;
                }
            }

            // Print if prime and greater than 1
            // 1 is not prime, and isPrime must be true, so only primes get printed.
            if (isPrime && i > 1) {
                // Print the prime followed by a space so all primes stay on one line.
                System.out.print(i + " ");
            }
        }
    }
}
```

Series1.java

```
// Import Scanner: used to read text input typed on the keyboard.
import java.util.Scanner;

/**
 * Computes the sum of the series ( $i^2 / i!$ ) for  $i = 1$  to a user-specified
 * range. Each term is calculated as the square of  $i$  divided by the factorial of  $i$ .
 */
public class Series1 {
    public static void main(String[] args) {
        // Create a Scanner object that reads from System.in (the keyboard).
        Scanner sc = new Scanner(System.in);

        // Ask the user for the last value of  $i$  in the series.
        System.out.println("Please enter the range of the series : ");
        // Read the range as an integer.
        int range = sc.nextInt();

        // sum stores the running total of all the terms; it is a double because terms are divisions.
        double sum = 0;

        // Compute each term:  $i^2 / i!$ 
        // Outer loop: one iteration per value of  $i$  from 1 up to the range.
        for (int i = 1; i <= range; i++) {
            // exp is the square of  $i$ , the numerator of the term.
            double exp = i * i;
            // fact will hold the factorial of  $i$ , started at 1.
            double fact = 1;

            // Calculate factorial of  $i$ 
            // Multiply every number from 1 up to  $i$  together.
            for (int j = 1; j <= i; j++) {
                fact *= j;
            }

            // div is the current term:  $i$  squared divided by  $i$  factorial.
            double div = exp / fact;
            // Add the term to the running total.
            sum += div;
        }

        // Print the final sum of the whole series for the given range.
        System.out.println("Sum of the series for the range : " + range + " = " + sum);
    }
}
```

Series2.java

```
// Import Scanner: used to read text input typed on the keyboard.
import java.util.Scanner;

/**
 * Computes the sum of the alternating series 1 - 3 + 5 - 7 + ... up to a
 * user-specified odd number. The sign alternates: odd-numbered terms are
 * positive, even-numbered terms are negative.
 */
public class Series2 {
    public static void main(String[] args) {
        // Create a Scanner object that reads from System.in (the keyboard).
        Scanner sc = new Scanner(System.in);

        // Ask the user for the last odd number of the series.
        System.out.print("Enter the last odd number: ");
        // Read the range as an integer.
        int range = sc.nextInt();

        // Validate that the input is odd
        // If the number is even, it cannot be the last term of this series.
        if (range % 2 == 0) {
            // Tell the user they must enter an odd number.
            System.out.println("Please enter an odd number.");
            // Stop the program early since the input is invalid.
            return;
        }

        // Number of terms in the series
        // For an odd range end, there are (range + 1) / 2 odd terms in total.
        int m = (range + 1) / 2;

        // Sum = m if m is odd, Sum = -m if m is even (derived from arithmetic series)
        // A ternary selects the sign of the sum based on whether the term count is even or odd.
        int sum = (m % 2 == 0) ? -m : m;

        // Print the computed sum of the alternating series.
        System.out.println("Sum = " + sum);
    }
}
```

SimpleCalculator.java

```
// Import Scanner: used to read the two numbers and the operator typed on the keyboard.
import java.util.Scanner;

/**
 * A simple console-based calculator that reads two integers and an operator
 * (+, -, *, /) from the user and performs the corresponding arithmetic operation.
 */
public class SimpleCalculator {

    /**
     * Main method: asks the user for two numbers and an operator,
     * then performs the matching arithmetic operation using a switch statement.
     */
    public static void main(String[] args) {
        // Create a Scanner object that reads from System.in (the keyboard).
        Scanner sc = new Scanner(System.in);

        // Prompt the user to enter the first number.
        System.out.println("Please enter the first number : ");
        // Read the first integer typed by the user and store it in 'num1'.
        int num1 = sc.nextInt();
        // Prompt the user to enter the second number.
        System.out.println("Please enter the second number : ");
        // Read the second integer typed by the user and store it in 'num2'.
        int num2 = sc.nextInt();
        // Prompt the user to enter the arithmetic operator.
        System.out.println("Please enter the operator(+ , - , * , /) : ");
        // Read the next token and take only its first character as the operator.
        char op = sc.next().charAt(0);

        // Perform the selected operation using a switch statement
        switch (op) {
            case '+':
                // Addition: print the two numbers and their sum.
                System.out.println(num1 + " + " + num2 + " = " + (num1 + num2));
                break;
            case '-':
                // Subtraction: print the two numbers and their difference.
                System.out.println(num1 + " - " + num2 + " = " + (num1 - num2));
                break;
            case '*':
                // Multiplication: print the two numbers and their product.
                System.out.println(num1 + " * " + num2 + " = " + (num1 * num2));
                break;
            case '/':
                // Division: print the two numbers and their quotient.
                System.out.println(num1 + " / " + num2 + " = " + (num1 / num2));
                break;
            default:
                // The user typed something that is not +, -, * or /, so show an error message.
                System.out.println("Please enter the correct operator");
                break;
        }
    }
}
```

StrongNumber.java

```
/**
 * Checks whether a number provided as a command-line argument is a Strong number.
 * A Strong number is one where the sum of the factorials of its digits equals
 * the number itself (e.g., 145 = 1! + 4! + 5!).
 */
public class StrongNumber {
    public static void main(String cp[]) {

        // Convert the first command-line argument (a String) into an integer.
        int num = Integer.parseInt(cp[0]);
        // Print the number that was supplied so the output shows the input.
        System.out.println("Num = " + num);

        // original keeps a working copy of the number so we can strip digits without losing num.
        int original = num;
        // sum accumulates the factorials of all the digits, started at 0.
        int sum = 0;

        // Process each digit of the number
        // Keep looping until every digit has been peeled off the number.
        while (original > 0) {
            // The modulo operator gives the units digit at each step.
            int digit = original % 10;
            // fact will hold the factorial of the current digit, started at 1.
            int fact = 1;

            // Calculate factorial of the current digit
            // Multiply everything from 1 up to the digit together.
            for (int i = 1; i <= digit; i++) {
                fact = fact * i;
                // Print the running factorial value as the multiplication progresses.
                System.out.println("Fact in = " + fact);
            }

            // Print the final factorial of the current digit.
            System.out.println("Fact = " + fact);
            // Add this digit's factorial to the running total.
            sum += fact;
            // Print the running total so far.
            System.out.println("Sum = " + sum);

            // Remove the current units digit so the next digit can be processed.
            original = original / 10;
        }

        // Check if sum of digit factorials equals the original number
        if (sum == num) {
            // The sum matches, so the number satisfies the Strong number rule.
            System.out.println("Strong number");
        } else {
            // The sum does not match, so it is not a Strong number.
            System.out.println("Not strong number");
        }
    }
}
```

SumOfDigits.java

```
// Import Scanner: used to read text input typed on the keyboard.
import java.util.Scanner;

/**
 * Reads an integer from the user and computes the sum of its digits.
 */
public class SumOfDigits {
    public static void main(String[] args) {
        // Create a Scanner object that reads from System.in (the keyboard).
        Scanner sc = new Scanner(System.in);
        // Ask the user to type the number whose digits should be summed.
        System.out.println("Please enter the number to find its sum of digits : ");
        // Read the number typed by the user.
        int num = sc.nextInt();
        // temp keeps a copy of num so we can peel off digits without losing the original.
        int temp = num;
        // sum accumulates the digits as they are extracted, started at 0.
        int sum = 0;

        // Extract each digit using modulo and add it to sum
        // Keep looping until every digit has been removed from temp.
        while (temp > 0) {
            // The modulo operator gives the units digit at each step.
            int digit = temp % 10;
            // Add the extracted digit to the running sum.
            sum += digit;
            // Remove the units digit by integer-dividing by 10.
            temp /= 10;
        }

        // Print the final total of all the digits.
        System.out.println("The sum of digits = " + sum);
    }
}
```

SwapWithoutVars.java

```
// Import Scanner: used to read text input typed on the keyboard.
import java.util.Scanner;

/**
 * Swaps two integer values without using a temporary variable.
 * Uses arithmetic operations (addition and subtraction) to perform the swap.
 */
public class SwapWithoutVars {
    public static void main(String[] args) {
        // Create a Scanner object that reads from System.in (the keyboard).
        Scanner sc = new Scanner(System.in);

        // Ask the user to type the first number.
        System.out.println("Please enter the first number : ");
        // Read the first number into variable a.
        int a = sc.nextInt();
        // Ask the user to type the second number.
        System.out.println("Please enter the second number : ");
        // Read the second number into variable b.
        int b = sc.nextInt();

        // Swap using arithmetic: a becomes sum, b becomes original a, a becomes original b
        // Step 1: a now holds the sum of both numbers.
        a = a + b;
        // Step 2: subtracting the new b gives the original a value, which is stored into b.
        b = a - b;
        // Step 3: subtracting the new b (original a) from the sum leaves the original b, stored into a.
        a = a - b;

        // Print both numbers after the swap to confirm they exchanged values.
        System.out.println("Numbers after swapping are A : " + a + " and B : " + b);
    }
}
```

VowelsConsonants.java

```
// Import Scanner: used to read text input typed on the keyboard.
import java.util.Scanner;

/**
 * Reads a word from the user and counts how many vowels (a, e, i, o, u)
 * and consonants it contains. The check is case-insensitive.
 */
public class VowelsConsonants {
    public static void main(String[] args) {
        // Create a Scanner object that reads from System.in (the keyboard).
        Scanner sc = new Scanner(System.in);

        // Array of vowel characters to check against
        char[] arr = {'a', 'e', 'i', 'o', 'u'};

        // Read the word typed by the user and immediately convert it to lowercase for a case-insensitive check.
        String text = sc.next().toLowerCase();
        // count tracks how many vowels were found, started at 0.
        int count = 0;

        // Count vowels by checking each character against the vowel array
        // Outer loop: visit every character of the word.
        for (int i = 0; i < text.length(); i++) {
            // Inner loop: compare the current character against every vowel.
            for (int j = 0; j < arr.length; j++) {
                // If the current character matches one of the vowels...
                if (text.charAt(i) == arr[j]) {
                    // ...then increment the vowel counter.
                    count++;
                }
            }
        }

        // Print the number of vowels that were found.
        System.out.println("Vowels = " + count);
        // Consonants = total letters minus the vowels; print that result.
        System.out.println("Consonants = " + (text.length() - count));
    }
}
```